

Design Patterns — Java Placement Notes

1. What is a Design Pattern?

A **Design Pattern** is a reusable and proven solution to a commonly occurring software design problem.

It is **not ready-made code**. It is a general idea or blueprint that tells us how to structure our code to solve a particular problem.

Why use Design Patterns?

- Make code reusable and maintainable.
- Reduce tight coupling between classes.
- Provide proven solutions to common problems.
- Make code easier to understand and extend.
- Give developers a common way to discuss software design.

3 Types of Design Patterns

Type	Purpose	Examples
Creational	Object creation	Singleton, Factory, Builder, Prototype
Structural	How classes/objects are connected	Adapter, Decorator, Facade
Behavioral	Communication between objects	Observer, Iterator, Strategy

Placement Point ★

Design Patterns are reusable solutions to commonly occurring software design problems.

2. Singleton Design Pattern

Definition

Singleton is a **Creational Design Pattern** that ensures that a class has **only one object/instance** in the application.

Examples where Singleton may be useful:

- Logger
- Configuration manager
- Cache manager
- Shared application resources

Basic Singleton

```
class Samosa {  
  
    private static Samosa samosa;  
  
    private Samosa() {  
    }  
  
    public static Samosa getSamosa() {  
  
        if (samosa == null) {  
            samosa = new Samosa();  
        }  
  
        return samosa;  
    }  
}
```

Usage:

```
Samosa s1 = Samosa.getSamosa();  
Samosa s2 = Samosa.getSamosa();  
  
System.out.println(s1 == s2);
```

Output:

```
true
```

Both variables refer to the same object.

3 Important Rules

1. Private Constructor

```
private Samosa() {}
```

Prevents direct object creation:

```
new Samosa(); // Not allowed
```

2. Static Instance

```
private static Samosa samosa;
```

Stores the single instance.

3. Static Getter Method

```
public static Samosa getSamosa()
```

Provides access to the instance.

Thread-Safe Singleton

In a multithreaded application, two threads may enter this condition at the same time:

```
if (samosa == null)
```

Both could create separate objects.

One solution is **Double-Checked Locking**:

```
class Samosa {  
  
    private static volatile Samosa samosa;  
  
    private Samosa() {}  
  
    public static Samosa getSamosa() {  
  
        if (samosa == null) {  
  
            synchronized (Samosa.class) {  
  
                if (samosa == null) {  
                    samosa = new Samosa();  
                }  
            }  
        }  
  
        return samosa;  
    }  
}
```

```
}  
}
```

Placement Point ★

Singleton ensures only one instance of a class. It commonly uses a private constructor, static instance, and public static getter method.

3. Three Ways to Break Singleton

A Singleton can mainly be broken using:

1. Reflection
2. Serialization
3. Cloning

1. Reflection

Reflection can access the private constructor.

```
Constructor<Samosa> c =  
    Samosa.class.getDeclaredConstructor();  
  
c.setAccessible(true);  
  
Samosa s2 = c.newInstance();
```

Now a second object can be created.

Solution

Protect the constructor or use an **Enum Singleton**.

```
enum Samosa {  
    INSTANCE;  
}
```

2. Serialization

Serialization saves an object as a byte stream.

Object → Serialization → File

Deserialization recreates it:

File → Deserialization → Object

Deserialization can create another Singleton object.

Solution: `readResolve()`

```
private Object readResolve() {  
    return getSamosa();  
}
```

This returns the existing Singleton instance.

3. Cloning

Cloning can create a copy:

```
Samosa s2 = (Samosa) s1.clone();
```

Now `s1` and `s2` may be different objects.

Solution

Prevent cloning or return the same instance.

```
@Override  
protected Object clone()  
    throws CloneNotSupportedException {  
  
    return getSamosa();  
}
```

Quick Revision

Problem	Solution
Reflection	Constructor protection / Enum

Problem	Solution
Serialization	<code>readResolve()</code>
Cloning	Prevent/override <code>clone()</code>

Remember:

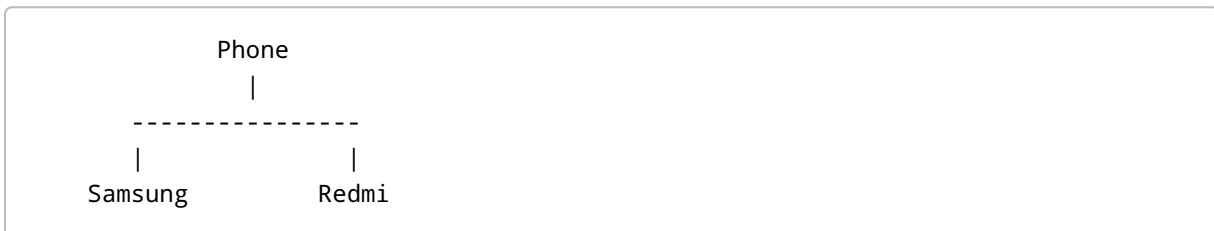
RSC = Reflection, Serialization, Cloning

4. Factory Design Pattern

Definition

Factory is a **Creational Design Pattern** that creates objects **without exposing object creation logic to the client**.

Suppose:



We want different objects based on input.

Parent Interface

```
interface Phone {

    void call();

}
```

Child Classes

```
class Samsung implements Phone {

    public void call() {
        System.out.println("Samsung Calling");
    }

}
```

```
}  
}
```

```
class Redmi implements Phone {  
  
    public void call() {  
        System.out.println("Redmi Calling");  
    }  
}
```

Factory

```
class PhoneFactory {  
  
    public static Phone getPhone(String type) {  
  
        if (type.equalsIgnoreCase("SAMSUNG"))  
            return new Samsung();  
  
        if (type.equalsIgnoreCase("REDMI"))  
            return new Redmi();  
  
        return null;  
    }  
}
```

Client:

```
Phone p =  
    PhoneFactory.getPhone("SAMSUNG");  
  
p.call();
```

Main Idea

Without Factory:

```
Phone p = new Samsung();
```

With Factory:

```
Phone p =  
    PhoneFactory.getPhone("SAMSUNG");
```

Remember:

Client asks → Factory creates → Client uses

Advantages

- Loose coupling
- Object creation logic is centralized.
- Client doesn't need to know creation details.
- Different objects can be returned based on input.

5. Abstract Factory Design Pattern

Definition

Factory creates objects. Abstract Factory manages/creates factories or families of related objects.

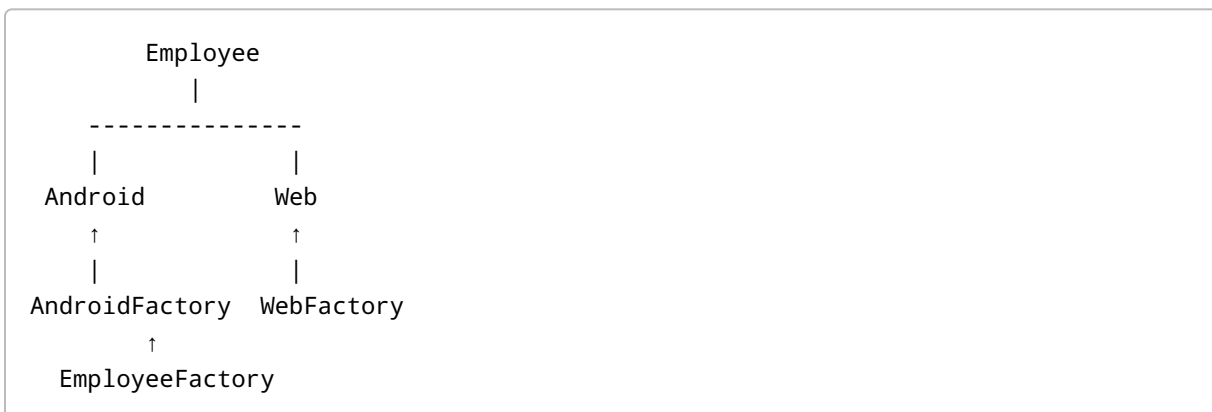
Abstract Factory is a **Creational Design Pattern** used to create **families of related objects** without specifying their concrete classes.

It is often called:

Factory of Factories

think if my group exists like employee , Staff , etc then who is responsible for calling which object so created factory and then internal factories

Suppose:



Example

```
abstract class Employee {  
  
    abstract void work();  
}
```

```
class AndroidDeveloper extends Employee {  
  
    void work() {  
        System.out.println("Developing Android App");  
    }  
}
```

```
class WebDeveloper extends Employee {  
  
    void work() {  
        System.out.println("Developing Website");  
    }  
}
```

Factories:

```
abstract class EmployeeFactory {  
  
    abstract Employee createEmployee();  
}
```

```
class AndroidFactory extends EmployeeFactory {  
  
    Employee createEmployee() {  
        return new AndroidDeveloper();  
    }  
}
```

Factory vs Abstract Factory

Factory	Abstract Factory
Creates required object	Creates families/groups of related objects
Simpler	More abstraction

Factory	Abstract Factory
One factory decision	Multiple related factories

Placement Point ★

Factory creates objects, while Abstract Factory provides an abstraction for creating families of related objects.

6. Builder Design Pattern

Definition

Builder is a **Creational Design Pattern** used to create complex objects **step by step**.

It is especially useful when an object has many optional properties.

Suppose:

```
class User {  
  
    String name;  
    int age;  
    String email;  
    String phone;  
    String address;  
}
```

Using a large constructor becomes confusing:

```
new User(  
    "Rohit",  
    22,  
    "abc@gmail.com",  
    "9999999999",  
    "Bangalore"  
);
```

Builder makes it readable:

```
User user = new UserBuilder()
    .setName("Rohit")
    .setAge(22)
    .setEmail("abc@gmail.com")
    .build();
```

Simple Builder

```
class UserBuilder {

    String name;
    int age;

    public UserBuilder setName(String name) {
        this.name = name;
        return this;
    }

    public UserBuilder setAge(int age) {
        this.age = age;
        return this;
    }

    public User build() {
        return new User(name, age);
    }
}
```

Notice:

```
return this;
```

This allows **method chaining**:

```
builder
    .setName("Rohit")
    .setAge(22)
    .build();
```

When to use?

When an object:

- Has many fields.
- Has many optional fields.
- Is complex to construct.
- Needs readable object creation.

Placement Point ★

Builder Pattern creates complex objects step by step and is useful when there are many optional parameters.

7. Prototype Design Pattern

Definition

Prototype is a **Creational Design Pattern** where new objects are created by **copying/cloning an existing object**.

Suppose creating an object requires:

```
Database call
API call
Complex calculations
```

Instead of repeating expensive creation:

```
Create Original Object
      ↓
      Clone
      /
Copy 1  Copy 2
```

Java Example

```
class NetworkConnection
    implements Cloneable {

    String ip;
```

```

    NetworkConnection(String ip) {
        this.ip = ip;
    }

    @Override
    protected Object clone()
        throws CloneNotSupportedException {

        return super.clone();
    }
}

```

Usage:

```

NetworkConnection n1 =
    new NetworkConnection("192.168.1.1");

NetworkConnection n2 =
    (NetworkConnection) n1.clone();

```

Advantages

- Faster when object creation is expensive.
- Avoids repeated initialization.
- Creates copies of existing objects.

Placement Point ★

Prototype Pattern creates new objects by cloning existing objects instead of creating them from scratch.

8. Shallow Copy vs Deep Copy

This is very important with the **Prototype Pattern**.

Suppose:

```

class Student {

    String name;
    Address address;
}

```

`Student` contains another object:

```
Student
|
Address
```

Shallow Copy

A new main object is created, but nested object references are shared.

```
Student 1 ---- student1 != student2
                → Same Address      But:
Student 2 ----/                  student1.address == student2.address
```

So if one changes the address object, the other may also see the change.

Java's default:

```
super.clone()
```



generally performs a **shallow copy**.

Deep Copy

Both the main object and nested mutable objects are copied.

```
Student 1 → Address 1      student1 != student2
                             student1.address != student2.address
Student 2 → Address 2
```

They are independent.

Difference

Shallow Copy	Deep Copy
Nested references are shared	Nested objects are copied
Faster	More work
Changes may affect both	Objects are independent

Shallow Copy

Deep Copy

`super.clone()` normally gives shallow copy Must usually be implemented manually

Placement Point ★

Shallow copy copies references of nested objects, while Deep Copy creates independent copies of nested mutable objects.

9. Implementing Deep Copy

Suppose:

```
class Address {  
  
    String city;  
  
    Address(String city) {  
        this.city = city;  
    }  
}
```

And:

```
class Student implements Cloneable {  
  
    String name;  
    Address address;  
}
```

For deep copy, clone the nested object too.

```
@Override  
protected Object clone()  
    throws CloneNotSupportedException {  
  
    Student student =  
        (Student) super.clone();  
  
    student.address =  
        new Address(this.address.city);  
}
```

```
    return student;  
}
```

Now:

```
Original Student  
  |  
Original Address  
  
Cloned Student  
  |  
New Address
```

Changing the cloned student's address won't change the original address object.

Other Deep Copy Methods

Deep copy can be implemented using:

- Manual copying
- Copy constructor
- Serialization/deserialization

Placement Point ★

In Deep Copy, every mutable referenced object that needs independence should also be copied.

10. Observer Design Pattern

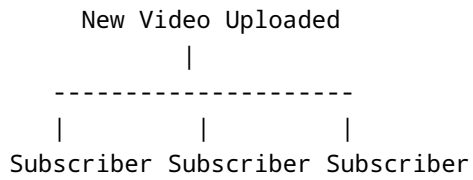
Definition

Observer is a **Behavioral Design Pattern** where **one object** notifies **multiple dependent objects** when its state changes.

Best real-life example:

YouTube Channel and Subscribers

```
YouTube Channel  
  |
```



The YouTube channel is the **Subject**.

Subscribers are **Observers**.

Observer

```
interface Observer {  
  
    void notified(String title);  
}
```

Subscriber

```
class Subscriber implements Observer {  
  
    String name;  
  
    Subscriber(String name) {  
        this.name = name;  
    }  
  
    public void notified(String title) {  
  
        System.out.println(  
            name + " notified: " + title  
        );  
    }  
}
```

Channel

```
class YouTubeChannel {  
  
    List<Observer> subscribers =  
        new ArrayList<>();  
  
    void subscribe(Observer o) {
```

```

        subscribers.add(o);
    }

    void uploadVideo(String title) {

        for (Observer o : subscribers) {
            o.notified(title);
        }
    }
}

```

Usage:

```

YouTubeChannel channel =
    new YouTubeChannel();

channel.subscribe(
    new Subscriber("Rohit")
);

channel.subscribe(
    new Subscriber("Amit")
);

channel.uploadVideo(
    "Java Design Patterns"
);

```

Both subscribers receive notifications.

Main Components

```

Subject
  ↓
Maintains observers
  ↓
State changes
  ↓
Notify()
  ↓
Observers updated

```

Use Cases

- YouTube subscriptions

- Notification systems
- Event listeners
- Stock price updates

Placement Point ★

Observer Pattern defines a one-to-many relationship where changes in one object automatically notify multiple dependent objects.

11. Iterator Design Pattern

Definition

Iterator is a **Behavioral Design Pattern** used to access collection elements one by one **without exposing the internal structure**.

Example:

```
Collection  
[10, 20, 30, 40]  
  
Iterator  
↓  
10 → 20 → 30 → 40
```

In Java:

```
Iterator<Integer> it =  
    list.iterator();  
  
while (it.hasNext()) {  
    System.out.println(  
        it.next()  
    );  
}
```

Two important methods:

```
hasNext()
```

Checks whether another element exists.

```
next()
```

Returns the next element.

Why Iterator?

The client doesn't need to know whether data internally uses:

```
Array  
ArrayList  
LinkedList  
Other collection
```

It simply traverses elements through an iterator.

Placement Point ★

Iterator Pattern provides a standard way to traverse elements of a collection without exposing its internal implementation.

12. Adapter Design Pattern

Definition

Adapter is a **Structural Design Pattern** that allows two incompatible interfaces to work together.

Best example:

Mobile Charger Adapter

Suppose:

```
Wall Socket → 220V  
Phone      → Needs lower voltage
```

An adapter sits between them:

```
220V
↓
Adapter
↓
Required Voltage
↓
Phone
```

In software:

```
Client
↓
Expected Interface
↓
Adapter
↓
Existing/Incompatible Class
```

Java Example

Suppose an old payment system has:

```
class OldPayment {

    void makePayment() {

        System.out.println(
            "Old Payment"
        );
    }
}
```

But our application expects:

```
interface Payment {

    void pay();
}
```

Create an Adapter:

```
class PaymentAdapter
    implements Payment {

    OldPayment oldPayment =
        new OldPayment();

    public void pay() {

        oldPayment.makePayment();
    }
}
```

Client:

```
Payment payment =
    new PaymentAdapter();

payment.pay();
```

The adapter converts the expected call:

```
pay()
```

into:

```
makePayment()
```

When to use?

- Existing class has an incompatible interface.
- Integrating old/legacy code.
- Integrating third-party libraries.
- We don't want to modify existing code.

Placement Point ★

Adapter Pattern acts as a bridge between two incompatible interfaces and allows them to work together.

Final Placement Cheat Sheet

Pattern	Type	Remember
Singleton	Creational	Only one object
Factory	Creational	Factory decides which object to create
Abstract Factory	Creational	Factory of factories / related object families
Builder	Creational	Build complex object step by step
Prototype	Creational	Create object by cloning
Observer	Behavioral	One change → notify many
Iterator	Behavioral	Traverse collection one by one
Adapter	Structural	Connect incompatible interfaces

Most Important Interview Questions

1. What is Singleton?

Ensures only one instance of a class.

2. How can Singleton be broken?

Reflection, Serialization, and Cloning.

3. What is Factory Pattern?

Creates the required object based on input while hiding object creation logic from the client.

4. Factory vs Abstract Factory?

Factory mainly selects/creates a product; Abstract Factory creates families of related products.

5. Why Builder?

To construct complex objects step by step, especially with many optional fields.

6. What is Prototype?

Creates new objects by cloning existing objects.

7. Shallow vs Deep Copy?

Shallow copy shares nested references; Deep Copy creates independent nested objects.

8. What is Observer?

When one object changes, multiple registered observers are notified.

9. What is Iterator?

Traverses collection elements without exposing internal structure.

10. What is Adapter?

Makes two incompatible interfaces work together.

Easy Memory Trick

Creational = Creating objects: Singleton, Factory, Abstract Factory, Builder, Prototype

Structural = Connecting objects: Adapter

Behavioral = Communication/behavior: Observer, Iterator